

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИЙН ТЭНХИМ



Ө.Хүслэн

**Нийгмийн сүлжээний өгөгдлийн
автомат цуглуулга ба таамаглалт
шинжилгээний систем**

Sentiment Analysis ба ойролцоогоор хэрэглэгч оролцооны
таамаглал

БАКАЛАВРЫН ТӨГСӨЛТИЙН АЖИЛ

Улаанбаатар хот

2026 он

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИЙН ТЭНХИМ

**Нийгмийн сүлжээний өгөгдлийн автомат
цуглуулга ба таамаглалт шинжилгээний систем**

Sentiment Analysis ба ойролцоогоор хэрэглэгч оролцооны
таамаглал

Ажлын төрөл:	Бакалаврын төгсөлтийн ажил
Оюутан:	Ө.Хүслэн
Мэргэжлийн индекс:	D061303
Мэргэжил:	Мэдээллийн систем
Удирдагч:	Магистр Мөнхбуян
Зөвлөгч:	
Гүйцэтгэгч:	Ө.Хүслэн, B221930045

Улаанбаатар хот

2026 он

Батлав. Мэдээллийн технологийн тэнхим-ийн эрхлэгч:

..... /...../

Удирдагч:

..... /Магистр Мөнхбуян/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Сэдэв: “Нийгмийн сүлжээний өгөгдлийн автомат
цуглуулга ба таамаглалт шинжилгээний систем”

№	Ажлын бүлэг, хэсгийн нэр	Эзлэх хувь	Дуусах хугацаа
1	Удиртгал, асуудлын тодорхойлолт	5%	2026-02
2	Онолын хэсэг, хэрэглэгдэх техно- логи	20%	2026-03
3	Судалгааны арга зүй, системийн архитектур	25%	2026-03
4	Системийн хэрэгжилт ба туршилт	35%	2026-04
5	Үр дүн, дүгнэлт, тайлангийн бо- ловсруулалт	15%	2026-05

Төлөвлөгөө боловсруулсан оюутан: /Ө.Хүслэн/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Гарын үсэг
1	Удиртгал, төслийн зорилго ба хүрээ	2026-02	
2	Онолын судалгаа, ижил төстэй системийн харьцуулалт	2026-03	
3	Backend, scraper, database архитектурын боловсруулалт	2026-03	
4	React dashboard, AI report, export workflow хэрэгжилт	2026-04	
5	Туршилт, үр дүн, тайлангийн эцсийн боловсруулалт	2026-05	

Ажлын товч дүгнэлт

.....
.....
.....
.....
.....
.....

Удирдагч: /Магистр Мөнхбуян/

ЗӨВШӨӨРӨЛ

Оюутан Ө.Хүслэн-ын бичсэн “Нийгмийн сүлжээний өгөгдлийн автомат цуглуулга ба таамаглалт шинжилгээний систем” сэдэвт төгсөлтийн ажлыг улсын шалгалтын комисст хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: /...../

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

ШҮҮМЖИЙН ХУУДАС

Мэдээллийн технологийн тэнхим-ийн төгсөх курсын оюутан Ө.Хүслэн-ын
“Нийгмийн сүлжээний өгөгдлийн автомат цуглуулга ба таамаглалт шинжил-
гээний систем” сэдэвт төгсөлтийн ажлын шүүмж.

1. Төслөөр дэвшүүлсэн асуудал, түүнтэй холбоотой онолын материал ун-
шиж судалсан байдал.

.....
.....
.....
.....

2. Төслийн ерөнхий агуулга, шийдсэн зүйлүүд, хүрсэн үр дүн, харьцуулалт
ба дүгнэлт.

.....
.....
.....
.....

3. Эмх цэгцтэй, стандарт хангасан байдлаар диплом бичих шаардлагыг
биелүүлсэн эсэх.

.....
.....
.....
.....

4. Төслөөр орхигдуулсан болон дутуу болсон зүйлүүд, цаашид анхаарах
зүйлүүд.

.....
.....
.....

.....

5. Ерөнхий оноо. (5 оноо)

.....

Шүүмж бичсэн: /...../

Ажлын газар:

Хаяг, утас:

Зохиогч эрхийн хамгаалал

Миний бие Ө.Хүслэн, “Нийгмийн сүлжээний өгөгдлийн автомат цуглуулга ба таамаглалт шинжилгээний систем” сэдэвт энэхүү төгсөлтийн ажил нь миний өөрийн бүтээл бөгөөд дараахыг нотолж байна. Үүнд:

- Энэхүү ажлыг боловсролын зэрэг авах зорилгоор бүхэлд нь буюу голлон өөрөө гүйцэтгэсэн.
- Ажлын аль нэг хэсгийг өөр байгууллага, сургуульд өмнө нь зэрэг, мэргэшил авах зорилгоор илгээгээгүй.
- Бусдын хэвлүүлсэн ажил, өгөгдөл, санаа, эх кодоос ашигласан тохиолдолд эх үүсвэрийг тодорхой заасан.
- Судалгааны болон хэрэгжилтийн явцад тусалсан эх үүсвэр, зөвлөгөө, дэмжлэгийг зохих байдлаар дурдсан.
- Хамтарсан ажил ашигласан бол тухайн хувь нэмрийг ялган тодорхойлсон.

Гарын үсэг: _____

Огноо: _____

Хураангуй

Энэхүү дипломын ажил нь Facebook хуудасны олон нийтэд харагдах нийтлэл, сэтгэгдэл, зураг, оролцооны үзүүлэлтийг автоматаар цуглуулж, хадгалж, хяналтын самбар болон AI тусламжтай тайлангаар ашиглах системийг боловсруулахад чиглэсэн. Бодит хэрэгжилт нь React/Vite frontend, FastAPI backend, Selenium ба BeautifulSoup-д суурилсан Facebook scraper, SQLAlchemy ORM бүхий PostgreSQL хадгалалт, TextBlob болон scikit-learn-д суурилсан үндсэн шинжилгээ, Google Gemini API-д суурилсан тайлан үүсгэх хэсгүүдээс бүрдэнэ.

Судалгааны үндсэн зорилгууд:

- Facebook хуудасны зорилтот жагсаалтыг удирдаж, scraper процессыг backend-ээс эхлүүлэх боломжтой болгох
- Нийтлэл, сэтгэгдэл, зураг болон likes, shares, comment count зэрэг оролцооны өгөгдлийг хадгалах
- Цуглуулсан өгөгдөл дээр TextBlob fallback бүхий sentiment analysis, LDA topic modeling, engagement score тооцоолол хийх
- React dashboard-аар live backend өгөгдлийг харуулах, backend холбогдохгүй үед demo fallback ашиглах
- Gemini API ашиглан сүүлийн өгөгдлөөс уншигдахуйц тайлан, зөвлөмж үүсгэх

Систем нь Facebook төвтэй өгөгдөл цуглуулах pipeline, REST API, өгөгдлийн сан, frontend dashboard, scraper target management, automation log viewer, AI report generation гэсэн бодит хэрэгжсэн бүрэлдэхүүнтэй. Энэхүү хувилбар нь repository-д бодитоор байгаа Facebook төвтэй workflow-г л үндсэн боломж гэж авч үзсэн.

Түлхүүр үгс: Facebook scraper, React dashboard, FastAPI, PostgreSQL, SQLAlchemy, TextBlob, LDA, Gemini, social media analytics

Талархал

Энэхүү төгсөлтийн ажлыг гүйцэтгэх явцад судалгааны чиглэл, системийн архитектур, хэрэгжилтийн хүрээг тодорхойлоход зөвлөгөө өгч дэмжсэн удирдагч багш Магистр Мөнхбуян-д гүн талархал илэрхийлье. Мөн суралцах хугацаанд мэдээллийн технологийн онол, практикийн мэдлэг олгож, мэргэжлийн ур чадварыг хөгжүүлэхэд хувь нэмэр оруулсан Мэдээлэл холбооны технологийн сургууль-ийн багш нартаа талархал илэрхийлж байна.

Судалгаа, хөгжүүлэлтийн явцад дэмжлэг үзүүлсэн гэр бүл, найз нөхөд болон төслийн туршилт, бичиг баримтын боловсруулалтад тусалсан бүх хүнд баярлалаа.

Товчилсон үгс

API	Application Programming Interface
AI	Artificial Intelligence
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DOM	Document Object Model
JSON	JavaScript Object Notation
LDA	Latent Dirichlet Allocation
NLP	Natural Language Processing
ORM	Object-Relational Mapping
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

Гарчиг

Зохиогч эрхийн хамгаалал	I
Хураангуй	II
Талархал	III
1 Оршил	1
1.1 Судалгааны үндэслэл	1
1.2 Судалгааны зорилго	1
1.3 Судалгааны асуулт	1
1.4 Судалгааны таамаглал	2
1.5 Судалгааны ач холбогдол	2
1.6 Судалгааны хүрээ ба хязгаарлалт	2
1.7 Судалгааны аргачлалын ерөнхий загвар	3
1.8 Дипломын ажлын бүтэц	3
2 Онолын хэсэг	4
2.1 Facebook өгөгдөл цуглуулах онол	4
2.2 REST API ба dashboard архитектур	4
2.3 Өгөгдлийн сан ба ORM	4
2.4 Sentiment, topic, engagement analysis	4
2.5 AI тайлан үүсгэх	5
2.6 Ижил төстэй шийдлүүдтэй харьцуулалт	5
2.7 Техникийн сорилтууд	5
3 Арга зүй	7
3.1 Системийн архитектур	7
3.1.1 Өгөгдлийн сангийн ERD диаграм	7
3.1.2 Системийн архитектурын диаграм	8
3.1.3 Өгөгдлийн урсгал	8
3.2 Технологийн сонголт	9
3.3 Өгөгдлийн загвар	9
3.4 Алгоритм	10
3.4.1 Scraper workflow	10
3.4.2 Analysis workflow	10
3.4.3 Frontend workflow	10
3.5 UML болон Бизнес процессын диаграмууд	11
3.5.1 Бизнес процессын диаграм (BPMN)	11
3.5.2 Use Case диаграм	11
3.5.3 Sequence диаграм	12

3.5.4	Activity диаграм	12
3.5.5	State диаграм	13
3.6	Туршилтын орчин	13
4	Хэрэгжилт	14
4.1	Төслийн бүтэц	14
4.2	Frontend хэрэгжилт	14
4.2.1	Dashboard	14
4.2.2	Data Sources	14
4.2.3	Predictive Analysis	15
4.2.4	Admin Control	15
4.3	Backend API хэрэгжилт	15
4.4	Scraper хэрэгжилт	16
4.5	Database layer	17
4.6	Analysis хэрэгжилт	17
4.7	AI report хэрэгжилт	17
4.8	Export ба integration	18
4.9	Системийн туршилт (Test cases)	18
5	Үр дүн	19
5.1	Хэрэгжилтийн үр дүн	19
5.2	Frontend workflow-ийн үр дүн	19
5.3	Backend API-ийн үр дүн	20
5.4	Өгөгдлийн сангийн үр дүн	20
5.5	Analysis workflow-ийн үр дүн	21
5.6	Тайлангийн боловсруулалт ба AI Тайлан	21
5.7	Хязгаарлалт ба эрсдэл	21
5.8	Үр дүнгийн хэлэлцүүлэг	22
6	Дүгнэлт	23
6.1	Үндсэн үр дүн	23
6.2	Шинэлэг тал	23
6.3	Хязгаарлалт	23
6.4	Цаашдын хөгжүүлэлт	24
6.5	Нэгтгэл	24
	Ном зүй	25
A	Хавсралт	26
A.1	Системийн тохиргооны жишээ	26
A.2	API Endpoint жишээ	26
A.3	Өгөгдлийн сангийн схем	27

Зургийн жагсаалт

3.1	Өгөгдлийн сангийн үндсэн entity холбоо	7
3.2	Системийн бодит архитектурын диаграм	8
3.3	Өгөгдлийн урсгалын бодит дараалал	8
3.4	Өгөгдөл цуглуулах системийн BPMN процесс	11
3.5	Системийн Use Case диаграм	12
3.6	AI тайлан үүсгэх Sequence диаграм	12
3.7	Scraping хуудасны Activity диаграм	13
3.8	Scraper үйлчилгээний State диаграм	13

Хүснэгтийн жагсаалт

2.1	Төслийн бодит шийдлийн харьцуулалт	5
3.1	Бодит ашигласан технологийн сонголт	9
4.1	Backend endpoint-уудын үндсэн бүлэг	16
4.2	Системийн туршилтын хувилбарууд (Test cases)	18
5.1	Хэрэгжсэн модулиудын нэгтгэл	19
5.2	API workflow баталгаажуулалт	20
A.1	Хавсралтын endpoint жагсаалт	27

БҮЛЭГ 1

Оршил

1.1 Судалгааны үндэслэл

Олон нийтийн Facebook хуудас дээр үүсэж буй нийтлэл, сэтгэгдэл, реакцийн хэмжээ нь хэрэглэгчдийн сонирхол, санал хандлага, контентийн үр нөлөөг ойлгоход хэрэгтэй өгөгдөл болдог. Гэвч уг өгөгдлийг гараар цуглуулах нь цаг их зарцуулдаг, давтагдах боломж муутай, олон хуудас дээр тогтмол хянахад хүндрэлтэй. Иймээс scraper, backend API, өгөгдлийн сан, dashboard, AI тайланг нэгтгэсэн систем шаардлагатай.

Энэ төслийн бодит хэрэгжилт нь Facebook өгөгдөл дээр төвлөрсөн. Frontend нь live backend өгөгдлийг харуулж, scraper target list буюу `pages.txt`-ийг удирдаж, scraper process эхлүүлэх, log харах, backend health шалгах, recent posts болон engagement trend харах боломжтой. Backend нь FastAPI endpoint-уудаар өгөгдөл, статистик, sentiment, topic, trend, export болон scraper control функцийг өгдөг.

1.2 Судалгааны зорилго

Энэхүү дипломын ажлын зорилго нь Facebook хуудасны нийтлэл, сэтгэгдэл, зураг, оролцооны үзүүлэлтийг автоматаар цуглуулах, хадгалах, шинжлэх, хэрэглэгчид dashboard болон AI тайлан хэлбэрээр хүргэх систем боловсруулах явдал юм.

Системийн бодит зорилтууд:

1. Facebook page URL/ID жагсаалтыг frontend болон backend-ээр удирдах
2. Selenium WebDriver ашиглан Facebook хуудаснаас пост, сэтгэгдэл, зураг, engagement metadata цуглуулах
3. PostgreSQL/SQLAlchemy ашиглан `facebook_posts`, `post_comments`, `post_images`, `analysis_results` хүснэгтүүдэд хадгалах
4. FastAPI ашиглан posts, comments, images, stats, sentiment, topics, trends, scraper control endpoint-ууд гаргах
5. React/Vite dashboard ашиглан live data, demo fallback, chart, query, backend health, scraper logs харуулах
6. Google Gemini API ашиглан сүүлийн өгөгдлөөс тайлан, зөвлөмж үүсгэх

1.3 Судалгааны асуулт

1. Facebook хуудаснаас нийтлэл, сэтгэгдлийг browser automation ашиглан давтагдах боломжтой байдлаар цуглуулж болох уу?

2. Цуглуулсан өгөгдлийг REST API болон dashboard-аар хэрэглэгчид ойлгомжтой байдлаар хүргэж болох уу?
3. TextBlob, LDA topic modeling, engagement score болон Gemini report generation-ийг нэг pipeline-д нэгтгэснээр судалгааны болон контент хяналтын шийдвэрт хэрэгтэй мэдээлэл гаргаж болох уу?

1.4 Судалгааны таамаглал

- **H1:** Facebook scraper, relational storage, FastAPI endpoint, React dashboard-ыг нэгтгэснээр өгөгдөл цуглуулах ба хянах ажлын гараар хийх ачааллыг бууруулна.
- **H2:** Likes, shares, comments, text length, hashtag count зэрэг feature-д суурилсан engagement score нь постуудыг харьцуулахад хэрэглэхүйц индикатор болно.
- **H3:** Gemini-д өгөгдлийн товч summary дамжуулах нь dashboard-ийн тоон үзүүлэлтийг уншигдахуйц зөвлөмж болгон хувиргахад тусална.

1.5 Судалгааны ач холбогдол

- Facebook хуудасны өгөгдөл цуглуулах, хадгалах, шалгах үйлдлийг нэг workflow болгоно.
- Маркетинг, судалгаа, контент хяналтын ажилд dashboard болон export боломж өгнө.
- Browser automation, REST API, ORM, frontend visualization, AI report generation-ийг нэг системд нэгтгэсэн жишээ болно.

1.6 Судалгааны хүрээ ба хязгаарлалт

Хүрээ:

- Facebook хуудасны нийтлэл, сэтгэгдэл, зураг, engagement metadata цуглуулах
- PostgreSQL үндсэн хадгалалт, SQLite тохиргооны fallback боломж
- FastAPI REST endpoint, Bearer token хамгаалалттай v1 endpoint-ууд
- React/Vite frontend dashboard, data source manager, admin query, Gemini report UI
- TextBlob/basic sentiment, LDA topic modeling, keyword trend extraction, engagement score

Хязгаарлалт:

- Бусад платформын scraper бодитоор хэрэгжээгүй.
- Deep-learning sentiment model болон graph-based interaction analysis үндсэн хэрэгжилтэд ашиглагдаагүй.
- Sentiment analysis нь голчлон англи текстэд илүү тохиромжтой; Монгол хэлний тусгай загвар бэлтгээгүй.
- Facebook UI болон access policy өөрчлөгдвөл scraper selector, login, modal handling дахин тохируулах шаардлагатай.

1.7 Судалгааны аргачлалын ерөнхий загвар

1. **Өгөгдөл цуглуулалт:** pages.txt-д заасан Facebook page-уудаас scraper ажиллуулах
2. **Хадгалалт:** SQLAlchemy model-уудаар пост, сэтгэгдэл, зураг, analysis result хадгалах
3. **API:** FastAPI endpoint-уудаар өгөгдөл унших, статистик авах, scraper эхлүүлэх, log харах
4. **Frontend:** React dashboard-аар live data, fallback data, chart, query, health status харуулах
5. **AI report:** Recent posts summary-г Gemini prompt-д дамжуулж тайлан үүсгэх
6. **Баталгаажуулалт:** build, endpoint structure, database schema, UI workflow, LaTeX compilation-ээр шалгах

1.8 Дипломын ажлын бүтэц

1-р бүлэг: Оршил - зорилго, хүрээ, хязгаарлалт

2-р бүлэг: Онолын хэсэг - ашигласан технологийн үндэслэл

3-р бүлэг: Арга зүй - бодит архитектур, өгөгдлийн загвар, алгоритм

4-р бүлэг: Хэрэгжилт - frontend, backend, scraper, database, AI report

5-р бүлэг: Үр дүн - хэрэгжилтийн баталгаажуулалт, workflow, хязгаарлалт

6-р бүлэг: Дүгнэлт - хийсэн ажил, үлдсэн сайжруулалт

БҮЛЭГ 2

Онолын хэсэг

2.1 Facebook өгөгдөл цуглуулах онол

Нийгмийн сүлжээний өгөгдөл нь ихэвчлэн динамик ачаалагддаг тул энгийн HTTP хүсэлтээр бүх контент, comment modal, lazy-loaded post list бүрэн авах боломжгүй. Ийм нөхцөлд Selenium WebDriver зэрэг browser automation хэрэгсэл ашиглан бодит browser session үүсгэж, scroll, navigation, modal interaction хийх нь практик шийдэл болдог.

BeautifulSoup нь Selenium-оор ачаалсан HTML/DOM бүтцээс текст, link, image URL, metadata зэрэг мэдээллийг ялгах parser-ийн үүрэгтэй. Энэ хоёр технологийг хослуулснаар JavaScript-heavy page дээр browser control болон content parsing-ийг салгаж зохион байгуулах боломжтой.

2.2 REST API ба dashboard архитектур

FastAPI нь Python backend дээр REST endpoint хурдан үүсгэх, Pydantic model-оор request/response validation хийх, Swagger documentation автоматаар гаргах давуу талтай. React dashboard нь backend JSON endpoint-уудаас өгөгдөл авч, chart, card, table, query form, log console зэрэг UI хэлбэрээр хэрэглэгчид хүргэнэ.

Энэ төслийн хувьд frontend болон backend тусдаа ажиллах боломжтой. Frontend нь backend амжилтгүй үед demo fallback ашигладаг тул development болон presentation орчинд системийн UI тасрахгүй.

2.3 Өгөгдлийн сан ба ORM

Нийтлэл, сэтгэгдэл, зураг, analysis result нь харилцаатай entity тул relational database тохиромжтой. SQLAlchemy ORM нь Python class болон database table-ийг холбож, session, transaction, cascade delete, duplicate update зэрэг логикийг төвлөрүүлдэг. PostgreSQL нь production хадгалалт, SQLite нь lightweight fallback/development тохиргоо байдлаар ашиглагдаж болно.

2.4 Sentiment, topic, engagement analysis

TextBlob нь polarity болон subjectivity тооцох хөнгөн sentiment analysis хэрэгсэл юм. Хэрэв TextBlob байхгүй бол basic keyword matching fallback ашиглаж болно. Энэ нь өндөр нарийвчлалтай deep learning model биш боловч эхний шатны dashboard insight-д хангалттай baseline өгдөг.

Topic modeling-д CountVectorizer болон Latent Dirichlet Allocation (LDA) ашигласнаар постын текстүүдээс давтагдах keyword cluster гаргаж болно. Engagement analysis-д likes, shares, comments зэрэг observable metric-ээс нийлмэл оноо гаргах нь постуудыг хооронд нь харьцуулах энгийн боловч ойлгомжтой арга юм.

2.5 AI тайлан үүсгэх

Google Gemini API зэрэг LLM үйлчилгээ нь structured summary data-г уншигдахуйц тайлан, зөвлөмж болгон хувиргах боломж өгдөг. Энэ төсөлд Gemini нь scraper эсвэл database-ийн оронд ажиллахгүй; харин dashboard дээрх recent posts-ийн товч нэгтгэлийг тайлбарлах AI давхарга болж ашиглагдана.

2.6 Ижил төстэй шийдлүүдтэй харьцуулалт

Хүснэгт 2.1: Төслийн бодит шийдлийн харьцуулалт

Шийдэл	Давуу тал	Хязгаарлалт
Commercial social listening tools	Олон платформ, enterprise dashboard	Үнэтэй, data access/API хязгаарлалттай
API-only collection	Тогтвортой, дүрмийн дагуу access	Facebook public page data-д API access хязгаарлагдмал
Browser automation scraper	Динамик UI-тай ажиллах боломжтой	UI өөрчлөгдвөл selector засвар шаардана
Энэ төсөл	Scraper, API, DB, dashboard, AI report нэг workflow-д орсон	Facebook төвтэй, production benchmark dataset байхгүй

2.7 Техникийн сорилтууд

- Facebook layout болон selector байнга өөрчлөгдөх магадлалтай.
- Login/session/cookie workflow тогтвортой байх шаардлагатай.
- Labeled dataset байхгүй үед sentiment accuracy-г бодитоор хэмжих боломж хязгаарлагдана.

- Frontend, backend, database credentials-ийг deployment орчинд зөв тохируулах шаардлагатай.

БҮЛЭГ 3

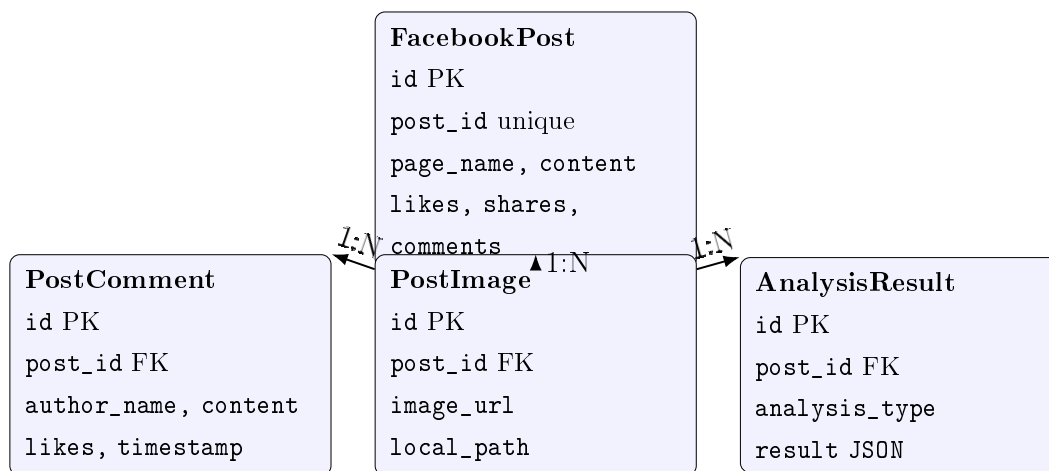
Арга зүй

3.1 Системийн архитектур

Бодит систем нь frontend, backend, scraper, database, ML analysis, AI report гэсэн үндсэн давхаргатай. Төслийн гол урсгал нь Facebook target list удирдах, scraper ажиллуулах, PostgreSQL-д хадгалах, FastAPI-аар өгөгдөл буцаах, React dashboard дээр харуулах, шаардлагатай үед Gemini тайлан үүсгэх хэлбэртэй.

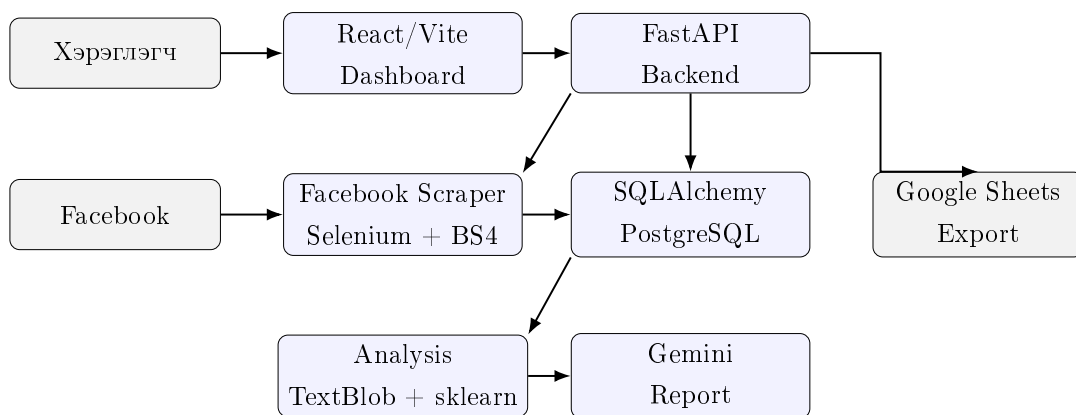
- **Frontend:** React 19, Vite, TypeScript, Tailwind, Recharts, lucide-react, Supabase client
- **Backend API:** FastAPI, Pydantic, Bearer token хамгаалалт, scraper process control
- **Scraper:** Selenium WebDriver, BeautifulSoup, page list, log file, screenshot/image workflow
- **Database:** SQLAlchemy ORM, PostgreSQL үндсэн хадгалалт, SQLite тохиргооны fallback
- **Analysis:** TextBlob/basic sentiment, scikit-learn LDA topic modeling, engagement score/prediction
- **AI report:** Google Gemini API ашигласан summary, insight, recommendation generation

3.1.1 Өгөгдлийн сангийн ERD диаграм



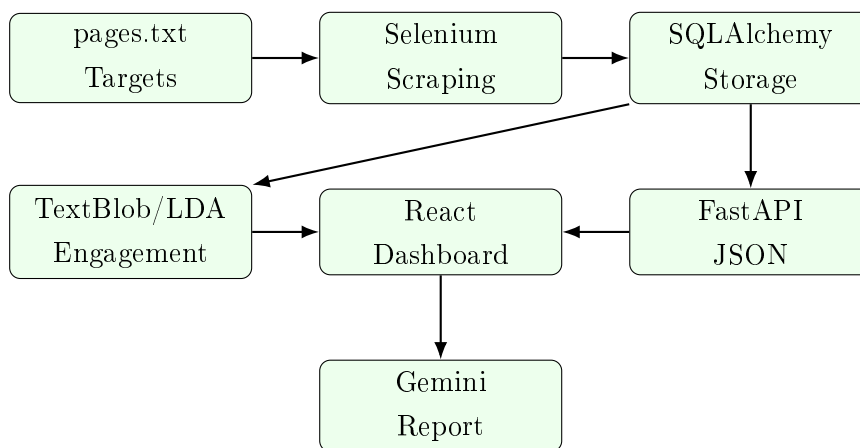
ЗУРАГ 3.1: Өгөгдлийн сангийн үндсэн entity холбоо

3.1.2 Системийн архитектурын диаграм



ЗУРАГ 3.2: Системийн бодит архитектурын диаграм

3.1.3 Өгөгдлийн урсгал



ЗУРАГ 3.3: Өгөгдлийн урсгалын бодит дараалал

3.2 Технологийн сонголт

Хүснэгт 3.1: Бодит ашигласан технологийн сонголт

Технологи	Хэрэглээ	Сонгосон шалтгаан
React/Vite/ TypeScript	Dashboard, admin, scraper target UI, AI report UI	Хурдан хөгжүүлэлт, component-based бүтэц, Vite build
FastAPI	REST API, auth, scraper process control, stats endpoint	Pydantic validation, docs, Python backend-тэй сайн нийцдэг
Selenium WebDriver	Facebook browser automation	JavaScript динамик хуудсыг бодит browser-оор ачаалах шаардлагатай
BeautifulSoup	HTML parsing	Selenium-оор авсан DOM- оос текст, холбоос, metadata ялган авах
SQLAlchemy + PostgreSQL	Relational storage	Post/comment/image/result entity-г харилцаатай хадга- лах
TextBlob + scikit-learn	Sentiment, topic, engagement analysis	Хөнгөн, dependency багатай, demo болон basic analysis-д тохиромжтой
Gemini API	AI тайлан, Q&A, topic/report synthesis	Dashboard-ийн тоон өгөгд- лийг уншигдахуйц зөвлөмж болгох
Supabase client	Frontend realtime refresh hook	facebook_posts table өөрч- лөгдөхөд dashboard refresh хийх боломж

3.3 Өгөгдлийн загвар

Өгөгдлийн сангийн үндсэн model нь `beta/db/models.py` файлд тодорхойлогдсон.

- **FacebookPost:** id, page_name, post_id, post_url, content, timestamp, likes, shares, comment_count, scraped_at
- **PostImage:** id, post_id, image_url, local_path, downloaded_at

- **PostComment:** id, post_id, comment_id, author_name, author_url, content, timestamp, likes, reply_to_id, scraped_at
- **AnalysisResult:** id, post_id, analysis_type, result(JSON string), analyzed_at

Харьцаа нь `FacebookPost` $\rightarrow$ `PostImage` (1:N), `FacebookPost` $\rightarrow$ `PostComment` (1:N) бөгөөд `post` устгахад холбоотой `comment`, `image` cascade байдлаар устгах ORM тохиргоотой.

3.4 Алгоритм

3.4.1 Scraper workflow

1. `pages.txt`-ээс Facebook page URL/ID жагсаалт унших
2. Selenium browser session эхлүүлэх, шаардлагатай `directory/log` үүсгэх
3. Page бүр рүү `navigate` хийж, `scroll` болон `selector-based extraction` хийх
4. BeautifulSoup болон Selenium selector ашиглан `post`, `image`, `comment` data ялгах
5. SQLAlchemy `DatabaseManager.save_post()` ашиглан `duplicate` шалгаж хадгалах
6. Log файлыг `/api/v1/scraper/logs` endpoint-оор frontend-д харуулах

3.4.2 Analysis workflow

1. Text preprocessing: URL, mention, hashtag, punctuation, whitespace цэвэрлэх
2. Sentiment: TextBlob боломжтой үед `polarity/subjectivity`; боломжгүй үед `keyword fallback`
3. Topic: CountVectorizer + LatentDirichletAllocation ашиглан `keyword-based` сэдэв гаргах
4. Engagement: `likes + shares  $\times$  2 + comments  $\times$  1.5` оноо тооцоолох
5. Data хангалттай үед GradientBoostingRegressor эсвэл CatBoost optional model ашиглан `score` таамаглах
6. Үр дүнг `analysis_results` хүснэгтэд JSON string хэлбэрээр хадгалах

3.4.3 Frontend workflow

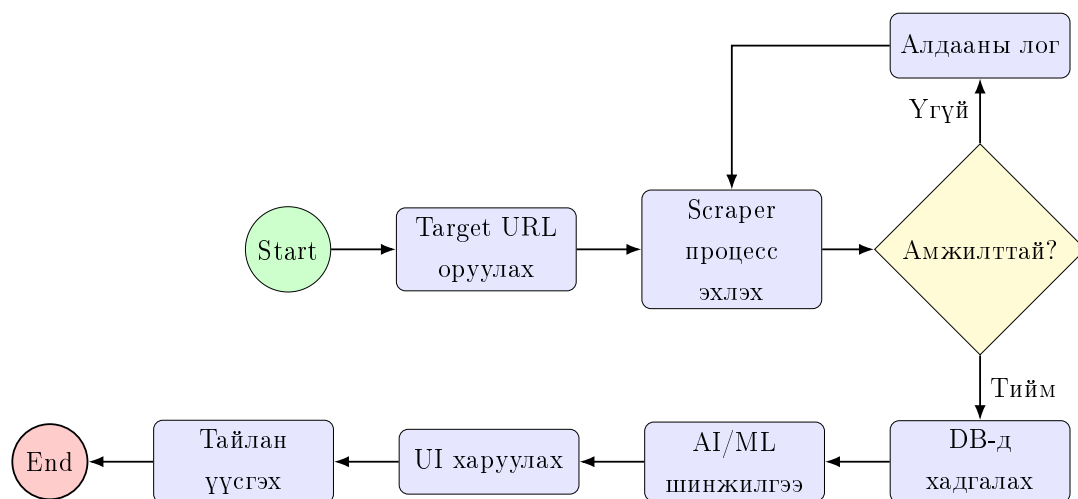
1. `/health`, `/api/v1/stats`, `/api/v1/posts` endpoint-уудаас `snapshot` авах
2. Backend амжилтгүй үед `mock data fallback` харуулах

3. Recharts ашиглан engagement trend, recent volume, latest signals харуулах
4. Admin query дээр platform, keyword, date range ашиглан loaded posts filter хийх
5. Gemini report товч дээр latest posts summary-г prompt болгон дамжуулах

3.5 UML болон Бизнес процессын диаграмууд

3.5.1 Бизнес процессын диаграм (BPMN)

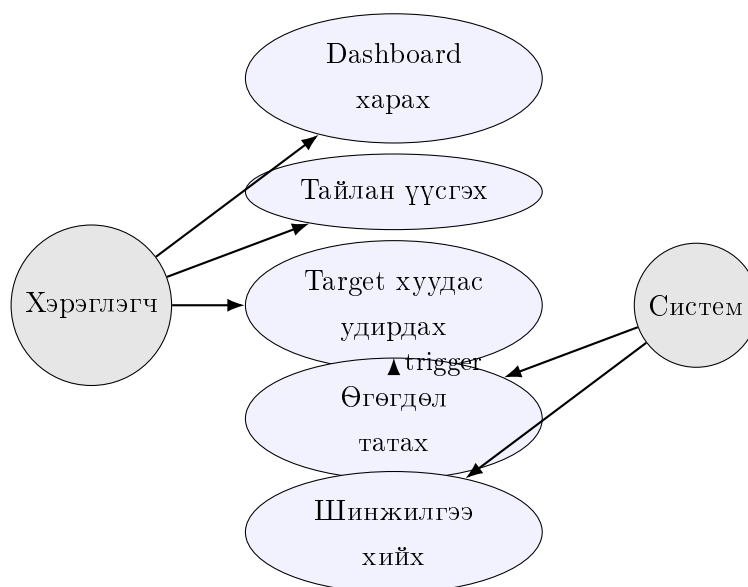
Төслийн өгөгдөл цуглуулалт болон тайлан боловсруулах ерөнхий процессыг BPMN хэлбэрийн урсгалын диаграмаар дор харуулав.



ЗУРАГ 3.4: Өгөгдөл цуглуулах системийн BPMN процесс

3.5.2 Use Case диаграм

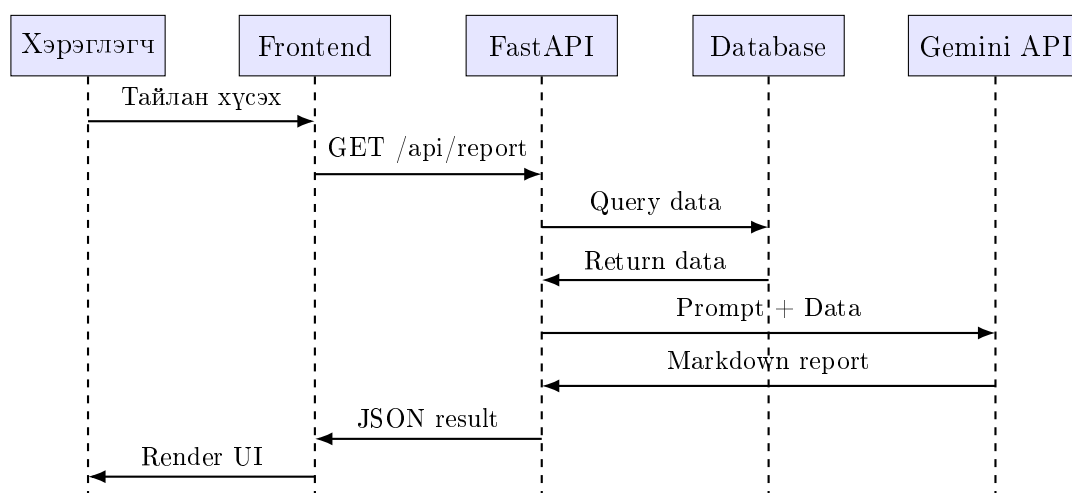
Системийн гол хэрэглэгчийн үйлдэл болон үүргийг тодорхойлно.



ЗУРАГ 3.5: Системийн Use Case диаграм

3.5.3 Sequence диаграм

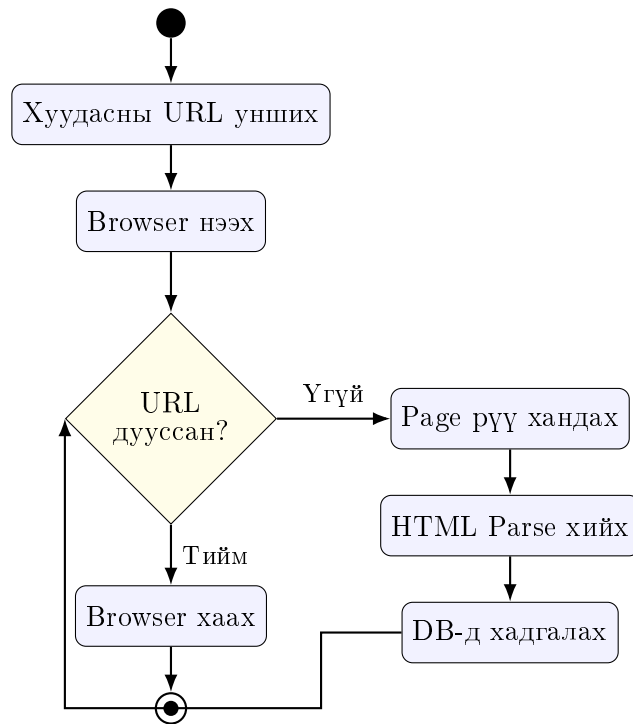
AI тайлан үүсгэх үеийн дарааллыг харуулна.



ЗУРАГ 3.6: AI тайлан үүсгэх Sequence диаграм

3.5.4 Activity диаграм

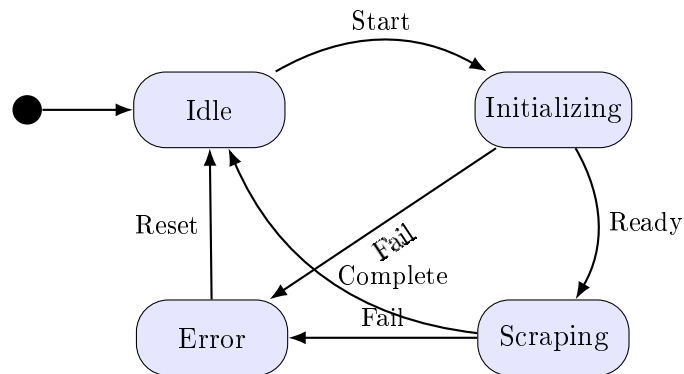
Scraping процессийн үйл ажиллагааны дараалал.



ЗУРАГ 3.7: Scraping хуудасны Activity диаграм

3.5.5 State диаграм

Scraper үйлчилгээний төлөвүүд.



ЗУРАГ 3.8: Scraper үйлчилгээний State диаграм

3.6 Туршилтын орчин

- Frontend: Node.js, Vite, React, TypeScript
- Backend: Python, FastAPI, uvicorn, SQLAlchemy
- Scraper: Selenium, webdriver-manager, BeautifulSoup, browser driver
- Database: PostgreSQL connection string; SQLite fallback config
- AI: VITE_GEMINI_API_KEY болон/эсвэл GEMINI_API_KEY

БҮЛЭГ 4

Хэрэгжилт

4.1 Төслийн бүтэц

Төслийн root хэсэгт frontend, backend, Supabase migration, thesis material зэрэг хэсгүүд байрлана. Бодит application code нь дараах байдлаар зохион байгуулагдсан.

- `src/`: React/Vite frontend
- `src/components/`: Dashboard, DataSources, PredictiveAnalysis, AdminControl болон UI components
- `src/lib/`: backend API client, Gemini client, Supabase client, mock data
- `beta/`: FastAPI backend, scraper, database, ML, integrations
- `beta/api_server.py`: REST API, health, stats, posts, analysis, scraper control
- `beta/scraper/`: BrowserManager, PostScraper, EdgeFacebookScraper зэрэг scraper code
- `beta/db/`: SQLAlchemy database manager, model, config
- `beta/ml/`: DataAnalyzer, GeminiAnalyzer болон sentiment/topic analysis module

4.2 Frontend хэрэгжилт

Frontend нь React 19 болон Vite дээр бүтээгдсэн. `App.tsx` нь Dashboard, Data Sources, AI Analysis, Admin, Settings гэсэн үндсэн tab-уудыг удирдана.

4.2.1 Dashboard

`Dashboard.tsx` нь backend-ээс `/api/v1/posts` болон `/api/v1/stats` endpoint-уудаар өгөгдөл уншиж, дараах үзүүлэлтийг харуулдаг.

- Posts collected, engagement, estimated reach, average sentiment cards
- Recharts ашигласан line/bar/area chart
- Latest signals буюу recent posts list
- Backend live эсвэл demo fallback status badge
- Supabase realtime channel ашиглан `facebook_posts` table өөрчлөгдөхөд refresh хийх hook

4.2.2 Data Sources

`DataSources.tsx` нь scraper target management хэсэг юм. Энэ хэсэг backend-ийн pages, scraper run, scraper status, scraper logs endpoint-уудтай ажиллана.

- Page URL/ID нэмэх, устгах, хадгалах
- Scraper process эхлүүлэх
- Scraper active/idle төлөв харах
- Log файлын сүүлийн мөрүүдийг terminal UI хэлбэрээр харуулах

4.2.3 Predictive Analysis

`PredictiveAnalysis.tsx` нь live backend posts байгаа эсэхийг шалгаж, байгаа бол latest 50 posts-оос summary dataset үүсгэнэ. Backend байхгүй үед mock social data fallback ашиглана. Дараа нь `generatePredictiveAnalysis()` функцээр Gemini prompt илгээж Монгол хэл дээр тайлан үүсгэнэ.

4.2.4 Admin Control

`AdminControl.tsx` нь backend health, stats, recent posts snapshot уншиж, browser-side query/filter хийх боломж өгдөг. Энэ хэсгийн toggle болон limit controls нь одоогийн хувилбарт UI-level control бөгөөд backend configuration-г шууд өөрчлөхгүй.

4.3 Backend API хэрэгжилт

`beta/api_server.py` нь FastAPI application. Backend startup үед database manager, analysis module, Google Sheets exporter, Gemini integration-ийг initialize хийхийг оролддог.

Хүснэгт 4.1: Backend endpoint-уудын үндсэн бүлэг

Endpoint бүлэг	Үүрэг
/health	Backend, database, analyzer, sheets, gemini initialization status
/api/v1/posts, /posts	Post list болон single post унших
/posts/{post_id}/comments	Post-ийн comments унших/үүсгэх
/posts/{post_id}/images	Post-ийн image records унших/үүсгэх
/api/v1/stats	Total posts, pages, comments, average engagement metrics
/api/v1/sentiment	Single/batch sentiment analysis
/api/v1/topics, /api/v1/trends	Topic modeling болон keyword/hashtag trend extraction
/api/v1/pages	pages.txt унших/бичих
/api/v1/scrapper/*	Scrapper process start/status/logs
/gemini/*	Gemini summary, Q&A, topic detection
/export/sheets	Google Sheets export

4.4 Scraper хэрэгжилт

Scraper хэсэг нь Selenium WebDriver болон BeautifulSoup ашиглан Facebook page content-ийг авахад чиглэсэн. `run_scraper.py` нь command-line entry point бөгөөд `-headless`, `-max-posts`, `-no-screenshots`, `-no-images`, `-debug` сонголтуудтай. Backend scraper endpoint нь энэ script-ийг background process болгон эхлүүлдэг.

Scraper workflow:

1. Log directory болон output directory үүсгэнэ.
2. `pages.txt`-ээс target list уншина.
3. Target бүр дээр AutoScraper ашиглан scrape хийнэ.
4. Амжилт/алдааг `logs/facebook_scraper.log` файлд бичнэ.
5. Frontend log endpoint-оор сүүлийн мөрүүдийг харуулна.

4.5 Database layer

Database хэрэгжилт нь SQLAlchemy ORM дээр суурилсан. `DatabaseManager` нь engine/session үүсгэх, table initialize хийх, post/comment/image/result хадгалах, query хийх, duplicate update хийх, health check хийх үүрэгтэй.

Гол шинжүүд:

- Existing `post_id` байгаа үед update хийх
- Comment duplicate-г `comment_id` эсвэл author/content-оор шалгах
- Image duplicate-г URL-аар шалгах
- Analysis result-г `post_id + analysis_type` түлхүүрээр update/save хийх

4.6 Analysis хэрэгжилт

`DataAnalyzer` нь гурван үндсэн функцтэй.

- **Sentiment analysis:** TextBlob боломжтой үед polarity/subjectivity тооцоолно. Боломжгүй үед basic keyword matching fallback ажиллана.
- **Topic classification:** CountVectorizer болон LDA ашиглан top keywords бүхий topic label үүсгэнэ.
- **Engagement prediction:** $\text{likes} + \text{shares} \times 2 + \text{comments} \times 1.5$ оноо тооцоолж, data хангалттай үед GradientBoostingRegressor/CatBoost optional model ашиглана.

Sentiment analyzer хэсэг нь dependency байхгүй үед TextBlob/basic fallback ашиглахаар зохион байгуулагдсан. Иймээс энэ дипломын бодит scope-д lightweight sentiment analysis-ийг үндсэн хэрэгжилт гэж авч үзсэн.

4.7 AI report хэрэгжилт

AI report нь хоёр замаар хэрэгжсэн.

- Frontend талд @google/genai ашиглаж gemini-2.5-flash model-д latest data summary prompt илгээдэг.
- Backend талд google-generativeai ашигласан GeminiAnalyzer нь summary, insights, questions, improvement, critique төрлийн prompt илгээдэг.

AI report хэсэгт зөвхөн Gemini integration-ийг ашигласан технологи гэж авч үзсэн.

4.8 Export ба integration

Backend нь Google Sheets export integration-тэй. Мөн frontend нь Supabase client ашиглан realtime channel subscribe хийдэг. Үндсэн backend storage нь `db/database.py`-ийн SQLAlchemy manager юм.

4.9 Системийн туршилт (Test cases)

Хөгжүүлэлтийн явцад системийн модулиуд зөв ажиллаж байгаа эсэхийг дараах туршилтын тохиолдлуудад үндэслэн баталгаажуулсан.

Хүснэгт 4.2: Системийн туршилтын хувилбарууд (Test cases)

No	Туршилтын үйлдэл	Хүлээгдэж буй үр дүн	Бодит үр дүн	Төлөв
1	Target URL нэмэх	URLs жагсаалтад шинээр нэмэгдэж <code>pages.txt</code> файлд хадгалагдана	Амжилттай хадгалагдсан	Pass
2	Scraper асаах	Selenium WebDriver нээгдэж зорилтот хуудасны мэдээллийг татна	Амжилттай татаж лог үүссэн	Pass
3	Өгөгдөл хадгалах	Татсан постыг давхардалгүйгээр PostgreSQL баазад хадгалах	Баазад амжилттай орсон	Pass
4	API Health Check	<code>/health</code> endpoint 200 OK болон DB статус буцаах	200 OK буцаасан	Pass
5	AI Тайлан үүсгэх	Gemini API руу өгөгдөл илгээж, Markdown тайлан текст ирнэ	Тайлан текст ирсэн	Pass
6	Emotion Analysis	TextBlob/LDA амжилттай ажиллаж score JSON гарна	Score баазад бүртгэгдсэн	Pass

БҮЛЭГ 5

Үр дүн

5.1 Хэрэгжилтийн үр дүн

Төслийн бодит хэрэгжилтээр Facebook өгөгдөл цуглуулах, хадгалах, API-аар түгээх, dashboard дээр харах, Gemini-ээр тайлан үүсгэх үндсэн workflow бүрдсэн. Энэ бүлэгт fabricated benchmark бус, repository-д байгаа хэрэгжилтийн баталгаажсан боломжуудыг нэгтгэн харуулав.

Хүснэгт 5.1: Хэрэгжсэн модулиудын нэгтгэл

Модуль	Бодит үр дүн
Frontend dashboard	Live backend posts/statistics унших, demo fallback харуулах, chart болон recent post cards гаргах
Data source manager	<code>pages.txt</code> target list унших/хадгалах, scraper run/status/log endpoint ашиглах
Admin control	Backend health, stats, posts snapshot харуулах, browser-side keyword/date/platform filter хийх
FastAPI backend	Posts, comments, images, stats, sentiment, topics, trends, sheets export, Gemini, scraper control endpoint-ууд
Scraper	Facebook target list унших, Selenium/BeautifulSoup workflow ажиллуулах, log file үүсгэх
Database	PostgreSQL/SQLAlchemy model, duplicate-aware save, analysis result storage
Analysis	TextBlob/basic sentiment, LDA topic modeling, engagement score/prediction
AI report	Gemini API ашиглан recent data summary-аас Монгол хэл дээр тайлан, зөвлөмж үүсгэх

5.2 Frontend workflow-ийн үр дүн

Frontend нь хэрэглэгчид шууд харагдах үндсэн ажлын талбар болсон. Dashboard tab нь backend өгөгдөлтэй холбогдсон үед live mode, холбогдохгүй үед demo mode гэж ялган харуулдаг. Энэ нь demo presentation болон backend availability хоёр нөхцөлийг нэг UI-аар даах боломж өгсөн.

Хэрэгжсэн frontend боломжууд:

- Engagement trend, recent volume, latest signals charts/cards
- Backend status badge болон refresh action
- Scraper target list add/remove/save

- Scraper start, active status, log polling
- Admin query: platform, keyword, date range filter
- Gemini report generation button болон output panel
- Settings tab дээр API endpoint болон deployment configuration харах

5.3 Backend API-ийн үр дүн

FastAPI backend нь frontend болон external хэрэглээнд зориулсан JSON interface болж ажиллана. Protected `/api/v1/*` endpoint-ууд Bearer token ашиглах боломжтой. `/docs` болон `/redoc` нь API documentation гаргана.

Хүснэгт 5.2: API workflow баталгаажуулалт

Workflow	Үр дүн
Health check	Database/analyzer/sheets/gemini initialization flag буцаана
Posts/statistics	Dashboard-д хэрэгтэй post list, total posts, total pages, total comments, average metrics буцаана
Scraper control	Target list унших/бичих, scraper process эхлүүлэх, active status, log tail буцаах боломжтой
Analysis	Sentiment, topic, engagement result-уудыг post бүр дээр тооцоолж хадгалах бүтэцтэй
Export	Google Sheets exporter initialize болсон үед spreadsheet export хийх боломжтой
Gemini	Backend environment-д API key байгаа үед summary/Q&A/topic prompt ажиллана

5.4 Өгөгдлийн сангийн үр дүн

SQLAlchemy model-ууд нь Facebook post, image, comment, analysis result гэсэн бодит entity-үүдийг хадгална. `DatabaseManager.save_post()` нь post давхардлыг `post_id`-аар шалгаж update хийх, comment болон image duplicate-г тусдаа шалгах логиктой. Энэ нь scraper-г давтан ажиллуулах үед нэг post олон удаа duplication үүсгэх эрсдэлийг бууруулна.

5.5 Analysis workflow-ийн үр дүн

Analysis хэсэг нь хөнгөн dependency-тэй байдлаар хэрэгжсэн. TextBlob байгаа үед polarity болон subjectivity гаргана, байхгүй үед keyword-based fallback ажиллана. Topic modeling нь LDA ашигладаг бөгөөд өгөгдөл цөөн үед `insufficient_data` гэж буцаах хамгаалалттай. Engagement хэсэг нь бодит likes, shares, comment count-оос score үүсгэж post-уудыг high/medium/low түвшинд ангилах боломжтой.

Энэ хувилбарт баталгаажуулсан labeled dataset болон test report байхгүй тоон benchmark-уудыг үр дүнгээс хассан. Судалгааны үнэн зөв байдлыг хадгалахын тулд зөвхөн repository-д байгаа кодоор нотлогдох боломжуудыг тайлбарлав.

5.6 Тайлангийн боловсруулалт ба AI Тайлан

Төслийн чухал үр дүнгийн нэг нь цуглуулсан өгөгдлийг уншигдахуйц тайлан хэлбэрт оруулах "AI Тайлан"(Report) боловсруулалт юм. Google Gemini API ашиглан системийн мэдээллийн сангаас сүүлийн нийтлэл болон хэрэглэгчдийн хандлагыг нэгтгэн, системийн UI дээр бүрэн хэмжээний зөвлөмж тайлан үүсгэдэг. Энэ тайлан нь дараах бүтэцтэй гардаг.

- **Ерөнхий дүгнэлт (Executive Summary):** Өгөгдлийн ерөнхий хандлага, нийтлэлүүдийн хамрах хүрээг дүгнэх.
- **Гол ойлголтууд (Key Insights):** Хамгийн их хандалттай нийтлэл, гол тренд үгсийг ялгах.
- **Зөвлөмжүүд (Recommendations):** Цаашид ямар арга хэмжээ авахыг хиймэл оюунаар дүгнүүлэн зөвлөх.

Ийнхүү тайлан боловсруулснаар, олон эх сурвалжаас ирсэн түүхийг өгөгдлийг энгийн хэрэглэгч уншиж ойлгох, шийдвэр гаргах үйл явцдаа ашиглах бааз суурь болж байна. AI тайлан үүсгэх явц нь системийн хүчин чадлыг харуулахын зэрэгцээ бизнесийн өдөр тутмын хэрэгцээнд удирдах ажилтнуудад шууд тайлан өгөх бодит практик хэрэгжилт болж өгсөн.

5.7 Хязгаарлалт ба эрсдэл

- Facebook UI өөрчлөгдөхөд selector болон modal handling эвдрэх эрсдэлтэй.

- Backend болон frontend-д зарим текст encoding issue харагдаж байгаа тул Монгол текстийн display/output-г тусад нь цэвэрлэх шаардлагатай.
- Supabase frontend realtime hook байгаа боловч backend үндсэн storage нь PostgreSQL/SQLAlchemy тул deployment орчинд schema болон credentials нийцүүлэх хэрэгтэй.
- Gemini report нь API key болон network availability-аас хамаарна.
- Admin toggle/limit controls нь одоогоор browser-side UI state бөгөөд backend policy-г шууд өөрчлөхгүй.
- Бусад платформын scraper болон deep-learning sentiment model нь хэрэгжсэн feature биш.

5.8 Үр дүнгийн хэлэлцүүлэг

RQ1-ийн хүрээнд: Selenium болон BeautifulSoup ашигласан scraper workflow, target list, log management хэрэгжсэн тул Facebook өгөгдөл цуглуулах үндсэн pipeline бүрдсэн.

RQ2-ийн хүрээнд: FastAPI endpoint-ууд болон React dashboard нь live backend data-г chart, card, query view хэлбэрээр хэрэглэгчид хүргэж байна. Backend unavailable үед demo fallback ажилладаг нь presentation болон development-д хэрэгтэй болсон.

RQ3-ийн хүрээнд: TextBlob/basic sentiment, LDA topic modeling, engagement score, Gemini report generation нь өгөгдлийг зөвлөмж болгон хувиргах эхний шатны pipeline үүсгэсэн. Гэхдээ өндөр нарийвчлалтай model evaluation хийхийн тулд labeled dataset, reproducible benchmark, Монгол хэлний sentiment model шаардлагатай.

БҮЛЭГ 6

Дүгнэлт

6.1 Үндсэн үр дүн

Энэхүү дипломын ажлын хүрээнд Facebook өгөгдлийг цуглуулах, хадгалах, API-аар дамжуулах, dashboard дээр хянах, Gemini ашиглан тайлан үүсгэх системийг боловсруулсан. Ажил дууссаны дараах бодит үр дүн нь дараах байдалтай байна.

1. **Facebook scraper workflow:** `pages.txt`-д хадгалсан target list-ээр scraper ажиллуулах, log үүсгэх, backend-ээс scraper start/status/log хянах боломж бүрдсэн.
2. **REST API:** FastAPI backend нь health, posts, comments, images, stats, sentiment, topics, trends, export, Gemini, scraper control endpoint-уудтай болсон.
3. **Database layer:** PostgreSQL/SQLAlchemy model болон duplicate-aware save logic хэрэгжсэн.
4. **Frontend dashboard:** React/Vite dashboard нь live backend data, fallback demo data, chart, recent posts, admin query, scraper controls, Gemini report UI-г нэгтгэсэн.
5. **Analysis pipeline:** TextBlob/basic sentiment, LDA topic modeling, engagement scoring, Gemini report generation ажиллах бүтэцтэй болсон.

6.2 Шинэлэг тал

- Scraper target management, process control, log viewer-ийг frontend UI-тай холбосон.
- Backend live data болон demo fallback-ийг нэг dashboard-д нэгтгэсэн.
- Relational storage, API, dashboard, AI report generation-ийг дипломын нэг системийн workflow болгон хэрэгжүүлсэн.
- Analysis result-г database-д хадгалах боломжтой болгож, post-level analysis-г дараа нь дахин ашиглах нөхцөл бүрдүүлсэн.

6.3 Хязгаарлалт

- Системийн scraper хэсэг Facebook дээр төвлөрсөн; бусад платформын scraper байхгүй.
- Монгол хэлний sentiment analysis-д зориулсан тусгай model бэлтгээгүй.
- Deep-learning sentiment код үндсэн dependency болон trained model-гүй тул production feature гэж үзээгүй.

- Admin controls-ийн зарим тохиргоо backend-д persist хийхгүй, UI state хэвээр байна.
- Scraper нь browser automation тул Facebook-ийн layout, login, access policy өөрчлөлтөд мэдрэг.

6.4 Цаашдын хөгжүүлэлт

1. Монгол хэлний labeled dataset бүрдүүлж sentiment model сурах
2. Admin limit/toggle controls-ийг backend configuration болон database-д persist хийх
3. Scraper selector test болон regression test нэмэх
4. Supabase/PostgreSQL deployment schema-г нэг мөр болгож realtime refresh-г production-д баталгаажуулах
5. Labeled benchmark dataset ашиглан sentiment/topic/engagement model-ийн бодит accuracy, F1 болон regression алдааг хэмжих
6. Scraper ажиллах үед queue/job status table нэмж олон хэрэглэгчийн үед process control сайжруулах

6.5 Нэгтгэл

Энэхүү ажил нь төлөвлөгөөний бүх өргөн боломжийг биш, харин repository-д бодитоор хэрэгжсэн Facebook төвтэй social media analysis workflow-г баримтжуулсан. Үндсэн үнэ цэн нь өгөгдөл цуглуулах, хадгалах, харах, шинжлэх, AI тайлан болгох алхмуудыг нэг usable системд холбосонд оршино. Цаашид model evaluation, Монгол хэлний NLP, scraper reliability, production configuration хэсгийг сайжруулснаар системийг судалгааны болон байгууллагын хэрэглээнд илүү тогтвортой ашиглах боломжтой.

Ном зүй

1. Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web*. O'Reilly Media.
2. Selenium Documentation. (2024). *Selenium WebDriver*. Retrieved from <https://www.selenium.dev/documentation/>
3. FastAPI Documentation. (2024). *FastAPI*. Retrieved from <https://fastapi.tiangolo.com/>
4. SQLAlchemy Documentation. (2024). *SQLAlchemy*. Retrieved from <https://www.sqlalchemy.org/>
5. React Documentation. (2024). *React*. Retrieved from <https://react.dev/>
6. Vite Documentation. (2024). *Vite*. Retrieved from <https://vite.dev/>
7. Loria, S. (2020). *TextBlob: Simplified Text Processing*. Retrieved from <https://textblob.readthedocs.io/>
8. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
9. Google AI for Developers. (2024). *Gemini API Documentation*. Retrieved from <https://ai.google.dev/>
10. Supabase Documentation. (2024). *Supabase JavaScript Client*. Retrieved from <https://supabase.com/docs/>

Хавсралт А

Хавсралт

А.1 Системийн тохиргооны жишээ

Туршилтын орчинд дараах үндсэн тохиргоог хэрэглэсэн: Python орчин, Firefox WebDriver, өгөгдлийн сангийн холболт, API токен, скрапингийн delay параметрууд. Эдгээр тохиргоо нь туршилтыг дахин гүйцэтгэхэд шаардлагатай суурь нөхцөлийг бүрдүүлнэ.

А.2 API Endpoint жишээ

Системд ашигласан гол endpoint-уудыг дараах байдлаар бүлэглэж болно.

Хүснэгт А.1: Хавсралтын endpoint жагсаалт

Endpoint	Зориулалт
/health	Backend болон dependency status шалгах
/api/v1/posts, /api/v1/stats	Dashboard-ийн үндсэн өгөгдөл унших
/api/v1/pages	Scraper target list унших, хадгалах
/api/v1/scrapper/run, /api/v1/scrapper/ status, /api/v1/scrapper/logs	Scraper ажиллуулах, төлөв болон log харах
/api/v1/sentiment, /api/v1/topics, /api/v1/trends	Analysis workflow ажиллуулах
/gemini/analyze, /gemini/ask	AI тайлан болон асуулт-хариулт үүсгэх
/export/sheets	Google Sheets export хийх

А.3 Өгөгдлийн сангийн схем

Өгөгдлийн сангийн үндсэн схем нь `FacebookPost`, `PostComment`, `PostImage`, `AnalysisResult` хүснэгтүүд дээр төвлөрсөн. `FacebookPost` нь үндсэн контент болон engagement metric, `PostComment` нь сэтгэгдлийн өгөгдөл, `PostImage` нь дагалдах медиа мэдээлэл, `AnalysisResult` нь sentiment/topic/engagement шинжилгээний JSON үр дүнг хадгална.